

Lab 05

Process Creation

Course: Operating System

Class: AI Section A and B

Instructor: Bilal Ahmed

Learning Objectives:

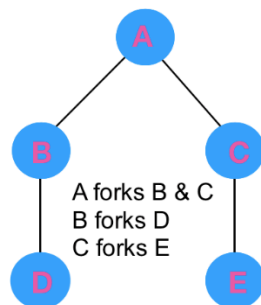
- Process Creation using fork.
- Understanding parent-child relationships between processes.
- Understanding and implementing zombie processes.

Introduction: Process creation in an operating system refers to the mechanism by which a new process is generated from an existing process. This process of creation involves allocating resources, initializing data structures, and setting up the execution environment for the new process.

Key aspects of process creation include:

- **Forking:** Parent process duplicates itself to create a child process.
- **Executing:** Child process loads a new program into its memory space and begins execution.
- **Resource Allocation:** Operating system allocates various resources to the new process.
- **Initialization:** OS initializes data structures and sets up execution environment for the new process.
- **Parent-Child Relationship:** Parent process controls creation and termination of child processes.
- **Process Termination:** OS manages termination of processes and releases allocated resources.

Forking: Implementing forking in C involves using the `fork()` system call, which creates a new process by duplicating the existing process.



Sample C code:

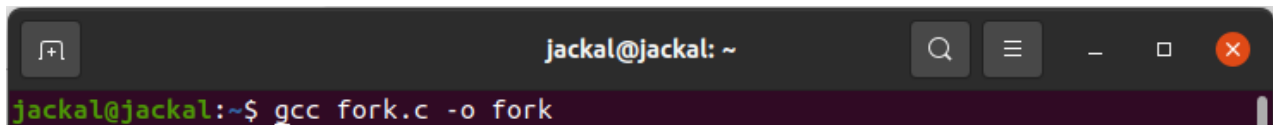
```
#include <stdio.h>
#include <unistd.h>

int main(){

printf("Before Forking \n");
fork();
printf("After Forking \n");
return 0;

}
```

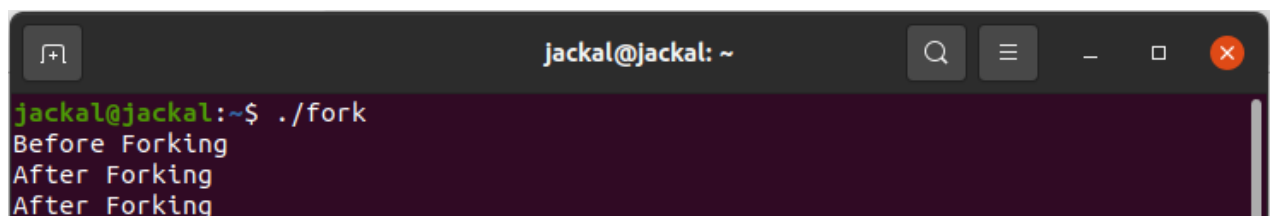
Compiling:

A terminal window with a dark background. The title bar shows 'jackal@jackal: ~'. The command 'gcc fork.c -o fork' has been entered and executed, resulting in a green prompt 'jackal@jackal:~\$'.

Header Files used in code.

- **<stdio.h>:** This is the standard input-output header file in the C programming language. It defines various functions and macros for performing input and output operations, such as printf(), scanf(), fprintf(), fscanf(), and so on. It also defines the FILE type and several standard streams like stdin, stdout, and stderr.
- **<unistd.h>:** This is a header file in the C standard library for the Unix operating system API. It provides access to various POSIX (Portable Operating System Interface) operating system services, such as process management (e.g., fork(), exec(), exit()), file manipulation (e.g., open(), close(), read(), write()), directory operations, memory management, and other system calls. It also defines symbolic constants, types, and macros related to system calls and process management.

Output.

A terminal window with a dark background. The title bar shows 'jackal@jackal: ~'. The command './fork' has been entered and executed. The output shows 'Before Forking' on the first line, followed by 'After Forking' on the second line, and another 'After Forking' on the third line, indicating two separate processes running.

Tracking it using PS command:

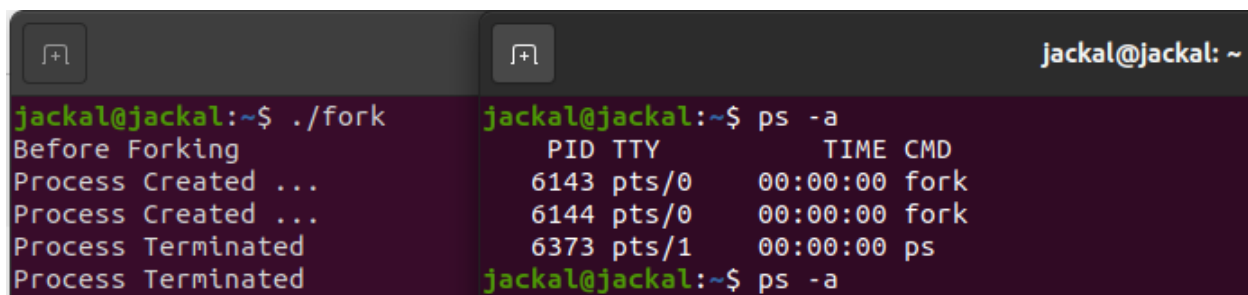
```
#include <stdio.h>
#include <unistd.h>

int main(){

printf("Before Forking \n");
fork();
printf("Process Created ...\n");
sleep(15);
printf("Process Terminated \n");
return 0;

}
```

Output



The image shows a terminal window with two panes. The left pane shows the output of the program: 'Before Forking', 'Process Created ...', 'Process Created ...', 'Process Terminated', and 'Process Terminated'. The right pane shows the output of the 'ps -a' command, which lists the processes: 'fork' (PID 6143) and 'ps' (PID 6373).

```
jackal@jackal:~$ ./fork
Before Forking
Process Created ...
Process Created ...
Process Terminated
Process Terminated

jackal@jackal:~$ ps -a
  PID TTY          TIME CMD
 6143 pts/0    00:00:00 fork
 6144 pts/0    00:00:00 fork
 6373 pts/1    00:00:00 ps
jackal@jackal:~$ ps -a
```

Fork return value:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid;

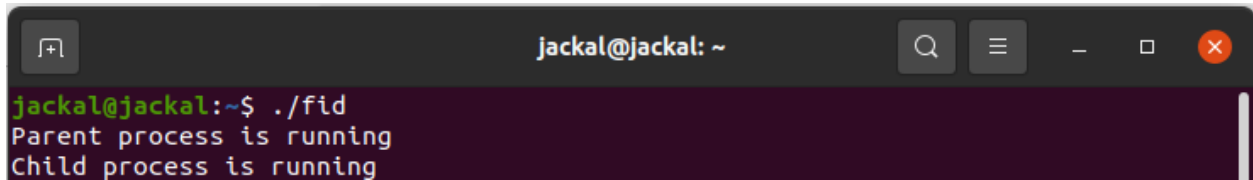
    // Fork a child process
    pid = fork();

    if (pid == 0) { // Child process
        printf("Child process is running\n ");
    }
    else { // Parent process
        printf("Parent process is running\n");
    }

    return 0;
}
```

```
}
```

Output:

A terminal window with a dark background and light-colored text. The title bar shows 'jackal@jackal: ~'. The prompt is 'jackal@jackal:~\$'. The command './fid' has been executed, resulting in two lines of output: 'Parent process is running' and 'Child process is running'.

Error Handling: Error handling is a programming technique used to manage and respond to errors that may occur during the execution of a program. In the provided code snippet, error handling is implemented for the `fork()` system call using the `perror()` function from the `<errno.h>` header file.

C Script.

```
#include <stdio.h>
#include <unistd.h>
#include <errno.h>

int main() {
    pid_t pid;

    // Fork a child process
    pid = fork();

    if (pid == -1) { // Error handling for fork failure
        perror("fork");
        return 1;
    }
    else if (pid == 0) { // Child process
        printf("Child process is running\n");
    }
    else { // Parent process
        printf("Parent process is running\n");
    }
}
```

PID and PPID:

- **PID (Process ID):** A unique numerical identifier assigned to each process by the operating system. It identifies a specific instance of a running program or process throughout its lifetime. It can be accessed using `getpid()` function.
- **PPID (Parent Process ID):** The PID of the parent process that created the current process. It identifies the immediate parent of a process in the process hierarchy. It can be accessed using `getpid()` function.

C Script:

```
#include <stdio.h>
#include <unistd.h>

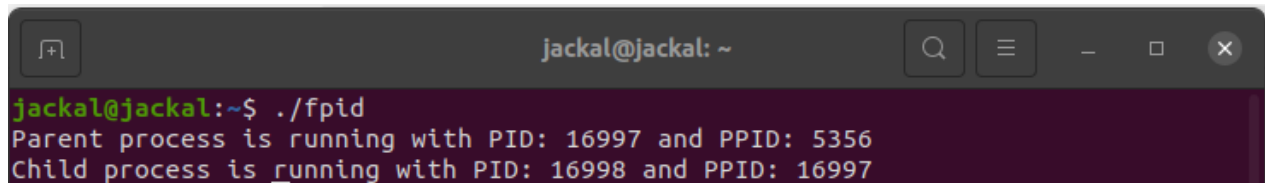
int main() {
    pid_t pid;

    // Fork a child process
    pid = fork();

    if (pid == 0) { // Child process
        printf("Child process is running with PID: %d and PPID: %d\n", getpid(), getppid());
    }
    else { // Parent process
        printf("Parent process is running with PID: %d and PPID: %d\n", getpid(), getppid());
    }
    sleep(15);
    return 0;
}
```

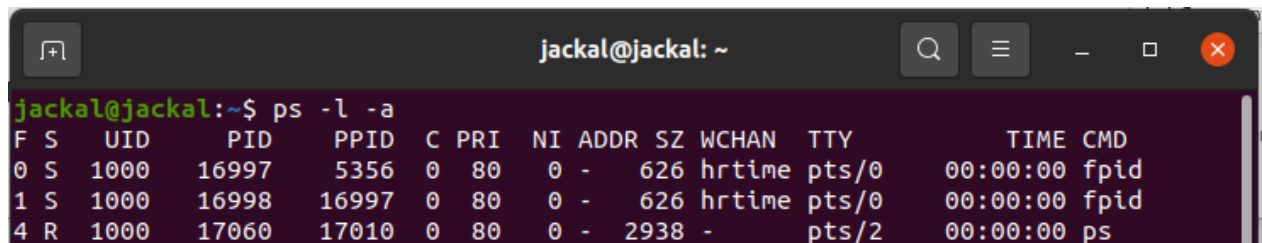
Output:

PID and PPID



```
jackal@jackal: ~
jackal@jackal:~$ ./fpid
Parent process is running with PID: 16997 and PPID: 5356
Child process is running with PID: 16998 and PPID: 16997
```

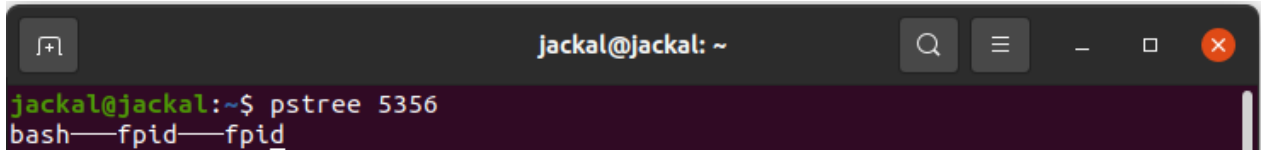
Checking using PS command:



```
jackal@jackal:~$ ps -l -a
```

F	S	UID	PID	PPID	C	PRI	NI	ADDR	SZ	WCHAN	TTY	TIME	CMD
0	S	1000	16997	5356	0	80	0	-	626	hrtimer	pts/0	00:00:00	fpid
1	S	1000	16998	16997	0	80	0	-	626	hrtimer	pts/0	00:00:00	fpid
4	R	1000	17060	17010	0	80	0	-	2938	-	pts/2	00:00:00	ps

Visualizing its parent-child relationship using pstree command:

A terminal window with a dark background. The title bar shows 'jackal@jackal: ~'. The command 'pstree 5356' has been executed, resulting in the output 'bash—fpid—fpid', which visually represents the parent-child relationship between the bash process and its child processes (fpid).

```
jackal@jackal:~$ pstree 5356
bash—fpid—fpid
```

Multiple Process:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    // Loop to create 5 child processes
    for (int i = 0; i < 5; i++) {
        pid_t pid = fork();

        if (pid == 0) { // Child process
            printf("Child process %d is running with PID: %d and PPID: %d\n", i+1, getpid(), getppid());
            break; // Exit the loop in the child process
        } else { // Parent process

            // Do nothing in the parent process, continue the loop
        }
    }

    sleep(15);
    return 0;
}
```

Output:

```
jackal@jackal: ~  
jackal@jackal:~$ ps -a  
  PID TTY          TIME CMD  
 22726 pts/0    00:00:00 test2  
 22727 pts/0    00:00:00 test2  
 22728 pts/0    00:00:00 test2  
 22729 pts/0    00:00:00 test2  
 22730 pts/0    00:00:00 test2  
 22731 pts/0    00:00:00 test2  
 22738 pts/1    00:00:00 ps  
jackal@jackal:~$ ps -a -l  
F S   UID        PID      PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD  
0 S   1000        22726     5356  0  80   0  -   593 hrtime pts/0    00:00:00 test2  
1 S   1000        22727     22726  0  80   0  -   626 hrtime pts/0    00:00:00 test2  
1 S   1000        22728     22726  0  80   0  -   626 hrtime pts/0    00:00:00 test2  
1 S   1000        22729     22726  0  80   0  -   626 hrtime pts/0    00:00:00 test2  
1 S   1000        22730     22726  0  80   0  -   626 hrtime pts/0    00:00:00 test2  
1 S   1000        22731     22726  0  80   0  -   626 hrtime pts/0    00:00:00 test2  
4 R   1000        22739    19404  0  80   0  -  2938 -       pts/1    00:00:00 ps  
jackal@jackal:~$ pstree 22726  
test2—5*[test2]
```

Zombie Process: A Zombie process is one that has finished its execution but remains in the system's process table, waiting for its parent process to retrieve its exit status.

Creating Zombie Process:

```
#include <stdio.h>  
#include <unistd.h>  
  
int main() {  
    // Loop to create 5 child processes  
    for (int i = 0; i < 5; i++) {  
        pid_t pid = fork();  
  
        if (pid == 0) { // Child process  
            printf("Child process %d is running with PID: %d and PPID: %d\n", i+1, getpid(), getppid());  
            break; // Exit the loop in the child process  
        }  
        sleep(5);  
    }  
  
    return 0;  
}
```

Output 1:

```
jackal@jackal:~$ ./test2  
Child process 1 is running with PID: 23021 and PPID: 23020  
Child process 2 is running with PID: 23022 and PPID: 23020  
Child process 3 is running with PID: 23023 and PPID: 23020  
Child process 4 is running with PID: 23024 and PPID: 23020  
Child process 5 is running with PID: 23025 and PPID: 23020
```

Output 2:

```
jackal@jackal: ~  
jackal@jackal:~$ ./test2  
Parent process is running with PID: 22932 and PPID: 5356  
Parent process is running with PID: 22932 and PPID: 5356  
Child process 1 is running with PID: 22933 and PPID: 22932  
Parent process is running with PID: 22932 and PPID: 5356  
Child process 2 is running with PID: 22934 and PPID: 22932  
Parent process is running with PID: 22932 and PPID: 5356  
Child process 3 is running with PID: 22935 and PPID: 22932  
Child process 4 is running with PID: 22936 and PPID: 22932  
Parent process is running with PID: 22932 and PPID: 5356  
Child process 5 is running with PID: 22937 and PPID: 22932
```

```
jackal@jackal: ~  
jackal@jackal:~$ ./multifpid  
Child process 1 is running with PID: 18970 and PPID: 18969  
Child process 2 is running with PID: 18972 and PPID: 18969  
Child process 3 is running with PID: 18974 and PPID: 18969  
Child process 4 is running with PID: 18977 and PPID: 18969  
Child process 5 is running with PID: 18980 and PPID: 18969  
jackal@jackal:~$
```

Continuous Monitoring.

```
jackal@jackal: ~  
jackal@jackal:~$ pstree 18969  
multifpid—3*[multifpid]  
jackal@jackal:~$ pstree 18969  
multifpid—3*[multifpid]  
jackal@jackal:~$ pstree 18969  
multifpid—4*[multifpid]  
jackal@jackal:~$ pstree 18969  
multifpid—5*[multifpid]  
jackal@jackal:~$ pstree 18969
```

Processes are zombies:

```
jackal@jackal:~$ top  
top - 14:56:10 up 4:34, 1 user, load average: 0.26, 0.29, 0.36  
Tasks: 315 total, 1 running, 309 sleeping, 0 stopped, 5 zombie  
%Cpu(s): 1.4 us, 0.8 sy, 0.0 ni, 97.7 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st  
MiB Mem : 63986.7 total, 57707.8 free, 3727.8 used, 2551.2 buff/cache  
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 59532.5 avail Mem
```



```
jackal@jackal: ~  
jackal@jackal:~$ ps -l -a  
F S  UID      PID     PPID  C  PRI  NI ADDR SZ WCHAN  TTY      TIME CMD  
0 S  1000    19884     5356  0   80   0 -   593 hrtime pts/0    00:00:00 multifpid  
1 Z  1000    19885     19884  0   80   0 -     0 -   pts/0    00:00:00  
1 Z  1000    19888     19884  0   80   0 -     0 -   pts/0    00:00:00  
1 Z  1000    19894     19884  0   80   0 -     0 -   pts/0    00:00:00  
1 Z  1000    19896     19884  0   80   0 -     0 -   pts/0    00:00:00  
1 Z  1000    19898     19884  0   80   0 -     0 -   pts/0    00:00:00  
4 R  1000    19901     19404  0   80   0 -   2938 -   pts/1    00:00:00 ps  
  
jackal@jackal:~$ ps -a  
PID TTY      TIME CMD  
20008 pts/0    00:00:00 multifpid  
20009 pts/0    00:00:00 multifpid <defunct>  
20011 pts/0    00:00:00 multifpid <defunct>  
20014 pts/0    00:00:00 multifpid <defunct>  
20015 pts/0    00:00:00 multifpid <defunct>  
20016 pts/0    00:00:00 multifpid <defunct>  
20017 pts/1    00:00:00 ps
```

Getting rid of zombies:

```
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main() {  
    // Loop to create 5 child processes  
    for (int i = 0; i < 5; i++) {  
        pid_t pid = fork();  
  
        if (pid == 0) { // Child process  
            printf("Child process %d is running with PID: %d and PPID: %d\n", i+1, getpid(), getppid());  
            break; // Exit the loop in the child process  
        }  
        wait(NULL);  
        sleep(5);  
    }  
    return 0;  
}
```

Explanation:

- After forking a child process, the parent process immediately calls `wait(NULL)`. This causes the parent process to wait for any of its child processes to terminate.
- Once a child process terminates, the parent process reaps its exit status and continues with the loop to create the next child process.
- Since the parent process waits for each child process to terminate before proceeding, no child process becomes a zombie.

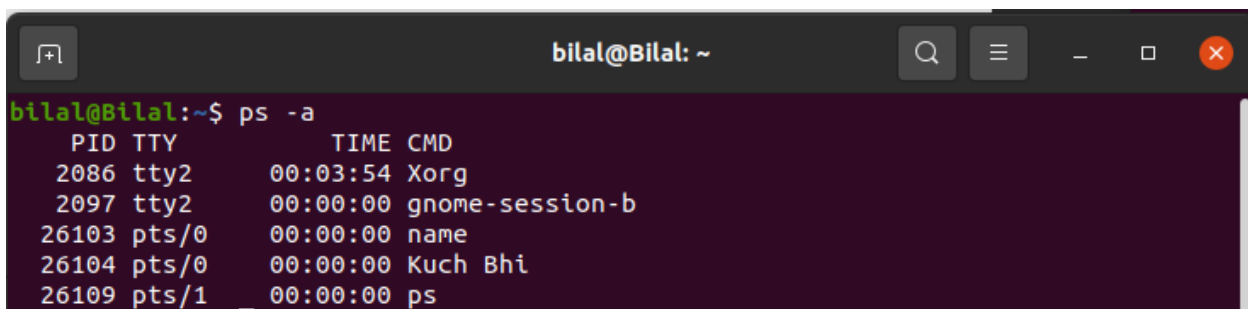
Naming the process. To specify a custom name for a process within a program, you can utilize the `prctl()` function, which is accessible through the `<sys/prctl.h>` header file. This function allows you to control various aspects of process behavior in the Linux environment. By using `prctl()`, you can set the name of the process to a specific value, enabling you to uniquely identify and manage it within the operating system.

C Script.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/prctl.h> // Include for prctl()

int main() {
    pid_t pid;
    pid = fork();
    if (pid == 0) { // Child process
        // Set custom name for the child process
        prctl(PR_SET_NAME, "Kuch Bhi", NULL, NULL, NULL);
        printf("Child process is running\n");
    }
    else { // Parent process
        printf("Parent process is running\n");
    }
    sleep(15);
    return 0;
}
```

Output:



```
bilal@Bilal: ~
bilal@Bilal:~$ ps -a
  PID TTY          TIME CMD
 2086 tty2        00:03:54 Xorg
 2097 tty2        00:00:00 gnome-session-b
26103 pts/0        00:00:00 name
26104 pts/0        00:00:00 Kuch Bhi
26109 pts/1        00:00:00 ps
```

Exercise:

Task 1: Write a Code which should name the parent and child processes as parent process and child process respectively.

Task 2: Create 3 processes with names; A, B and C respectively, where A should be parent of B and C. Show it using `pstree` and their PIDs.

Task 3: Explore `exec()` function and use it in a simple C program.

Task 4: Use fork in real-life example and explain it.